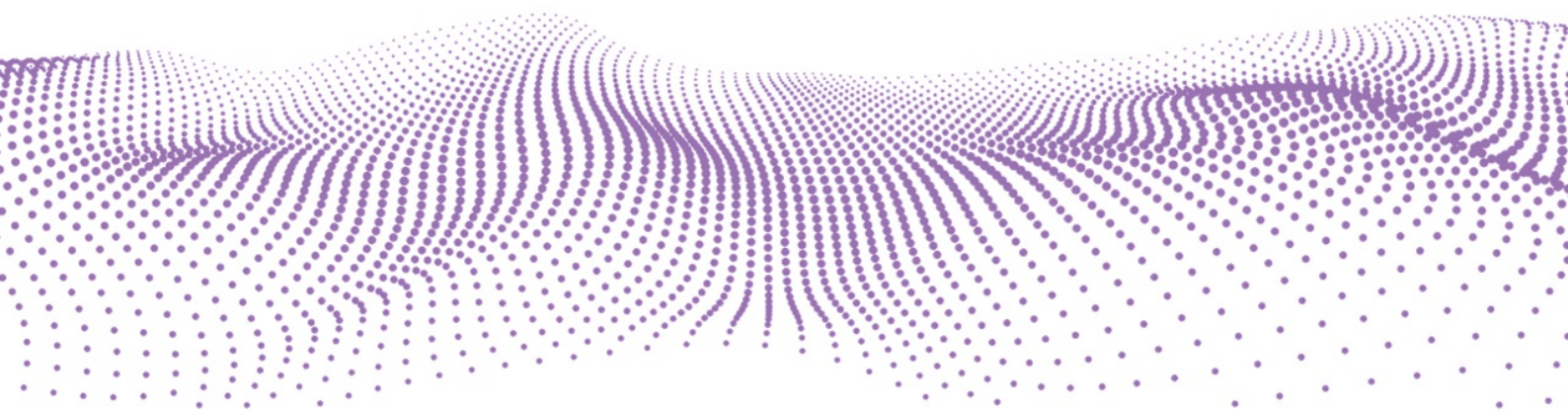


Black Duck Signal PoVガイド

2026/03/10

ブラック・ダック・ソフトウェア合同会社



Black Duck Signal PoVガイド

ブラック・ダック・ソフトウェア合同会社

2026/03/10

- [1 PoVガイド](#)
 - [1.1 目次](#)
 - [1.2 1. アーキテクチャー](#)
 - [1.2.1 1.1. Black Duck Signalとのやり取り](#)
 - [1.2.2 1.2. 解析プロセスとアーキテクチャー](#)
 - [1.3 2. ユースケースI | コードをアップロードして解析する](#)
 - [1.3.1 2.1. 解析フロー](#)
 - [1.3.2 2.2. 実行までの流れ](#)
 - [1.3.3 2.3. 準備 | アクセストークンの作成](#)
 - [1.3.4 2.4. インストール | Bridge CLIのインストール](#)
 - [1.3.5 2.5. 設定](#)
 - [1.3.6 2.6. Bridge CLIの実行](#)
 - [1.3.7 2.7. 解析結果の確認](#)
 - [1.4 3. ユースケースII | コーディングアシスタントと共に使う](#)
 - [1.4.1 3.1. 解析フロー](#)
 - [1.4.2 3.2. 解析モード](#)
 - [1.4.3 3.3. 共通準備](#)
 - [1.4.4 3.4.A. Visual Studio CodeのCopilotと共に使う](#)
 - [1.4.5 3.4.B. Visual Studio CodeのClaudeと共に使う](#)
 - [1.5 Appendix. リモート環境にMCPサーバーを追加する](#)

1 PoVガイド

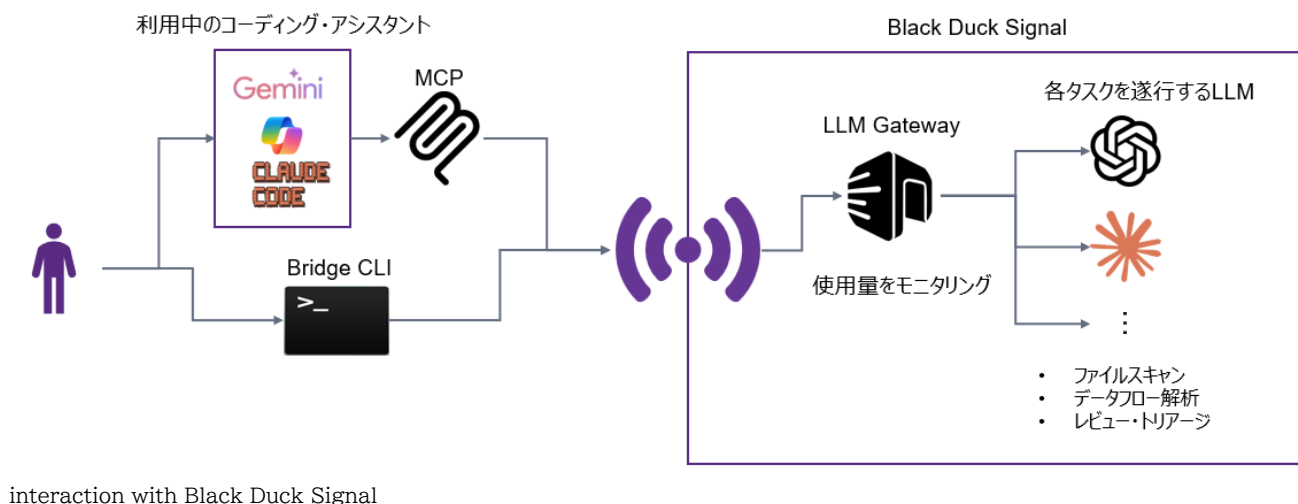
1.1 目次

1. アーキテクチャー
 - 1.1. Black Duck Signalとのやり取り
 - 1.2. 解析プロセスとアーキテクチャー
2. ユースケースI | コードをアップロードして解析する
 - 2.1. 解析フロー
 - 2.3. 実行までの流れ
 - 2.4. 準備
 - 2.5. インストール
 - 2.6. 実行
 - 2.7. 解析結果の確認
3. ユースケースII | コーディングアシスタントと共に使う
 - 3.1. 解析フロー
 - 3.2. 解析モード
 - 3.3. 共通準備
 - 3.4.A. Visual Studio CodeのCopilotと共に使う
 - 3.4.B. Visual Studio CodeのClaudeと共に使う

1.2 1. アーキテクチャー

1.2.1 1.1. Black Duck Signalとのやり取り

使用者はコーディングアシスタント、Bridge CLIを通じてBlack Duck Signalとやり取りします。サービス内部ではLLM Gatewayを通じて各タスクの実行に最適な各種LLMにルーティングします。



interaction with Black Duck Signal

1.2.2 1.2. 解析プロセスとアーキテクチャー

CLIから個々のファイルをLLMに送信して解析します。その後必要に応じてデータフロー解析エージェントが解析を実施して偽陽性を取り除きます。最後に偽陽性をさらに減らすためにオーバーサイトエージェントが詳細にレビューします。

1.2.2.1 ■ 解析プロセス

1. 単一ファイル解析

個々のファイルをLLMに送信して逐一解析

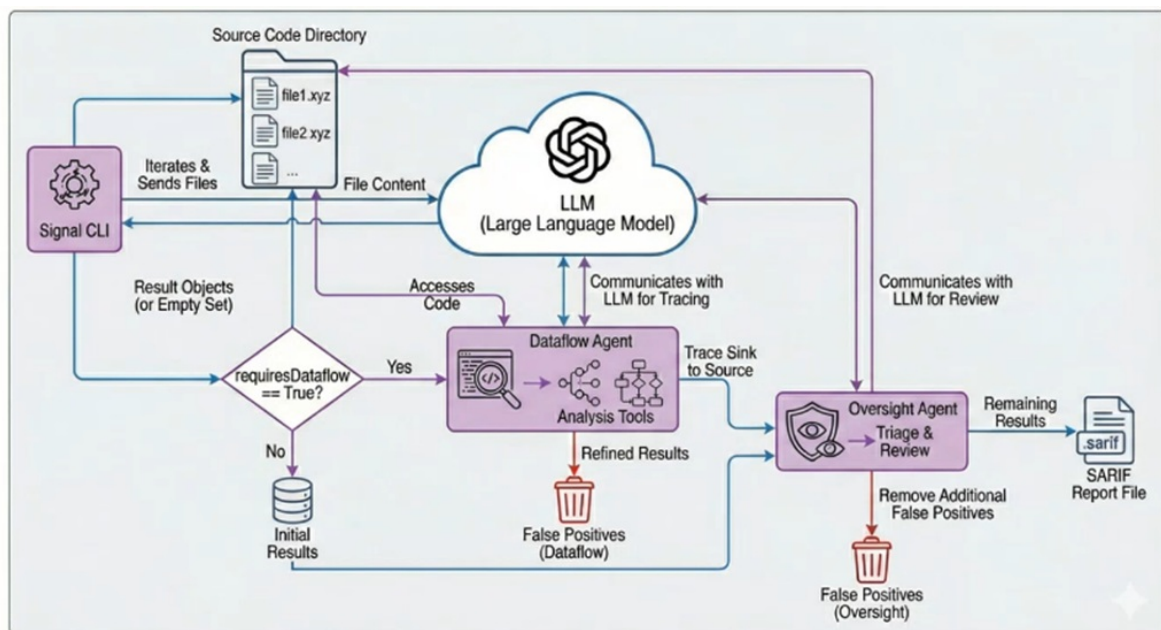
2. データフロー解析(必要に応じて)

データフロー解析が必要そうな怪しいも問題があれば専用エージェントがデータフロー解析を実施

3. レビュー/トリアージ

偽陽性のものを減らすように専用エージェントが最終チェックを行う

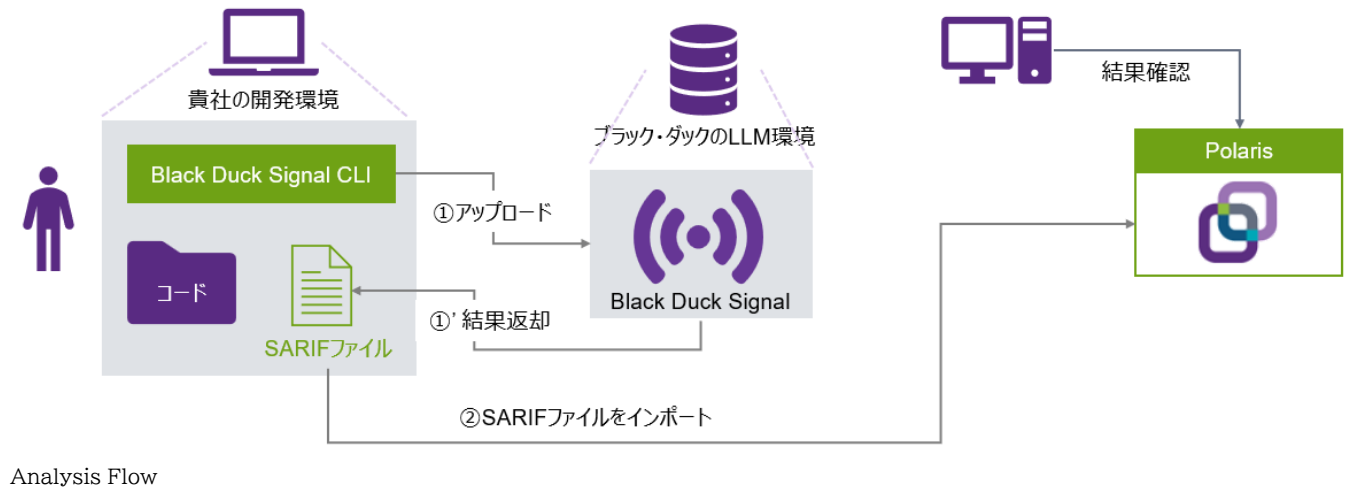
1.2.2.2 ■ アーキテクチャー



1.3 2. ユースケースI | コードをアップロードして解析する

1.3.1 2.1. 解析フロー

Black Duck Signal CLIからBlack Duck Signalにコードをアップロードしてスキャンします。SARIFで出力された解析結果をPolaris上にインポートして結果を確認します。



Analysis Flow

1.3.2 2.2. 実行までの流れ

通常のBridge CLIのインストールに加えて設定ファイル中のBlack Duck Signalの設定の記載が必要です

1. 準備

- LLM Gateway用のキーの作成 (事前にいただいた情報をもとにブラック・ダックが発行いたします)
- Polarisのアカウント設定 (Black Duck Signalのバイナリでの実行と同じため割愛)
- Polarisアクセストークンの作成 (Black Duck Signalのバイナリでの実行と同じため割愛)

2. インストール

- Bridge CLI(3.12.0+)のインストール
- Black Duck Signal CLIのディプロイ

3. 設定

- 環境変数の設定
- Bridge CLIの設定ファイル上でBlack Duck Signal用の設定を記述する

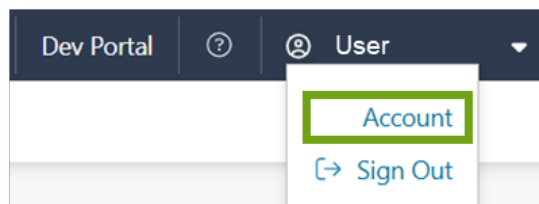
4. 実行

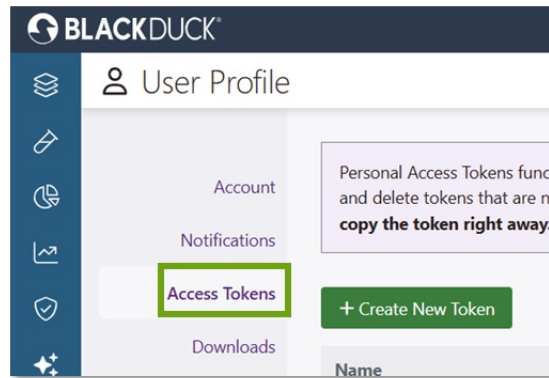
Bridge CLIを実行する

1.3.3 2.3. 準備 | アクセストークンの作成

Polarisアクセストークンをアカウント画面から作成します

[PolarisのPoC環境](#)にログインし、アカウント画面の「アクセストークン」タブからトークンを作成します

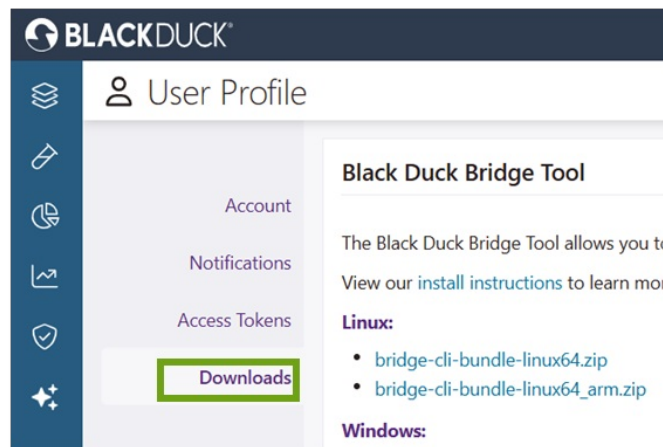




1.3.4 2.4. インストール | Bridge CLIのインストール

3.12.0以降のBridge CLIをダウンロードして実行可能なようにPATHに登録します

1. アカウントの「ダウンロード」タブから最新(3.12.0以降)のバージョンをダウンロードします



2. PATHに登録して実行可能にします

```
# Linuxの場合。お使いのOSに応じて変更してください
export POLARIS_BRIDGE_CLI_PATH=/your/install/bridge-cli-bundle-3.12.0-linux64
export PATH=${PATH}:${POLARIS_BRIDGE_CLI_PATH}
```

1.3.5 2.5. 設定

1.3.5.1 2.5.1. Black Duck Signalの設定ファイルの作成

プロジェクト内の任意の場所に設定ファイルを作成・記載します

1.3.5.1.1 ■ 設定ファイルの概要

- 目的: 実行時に設定を読み込む
- 要件: Black Duck Signalのバージョン >= 0.1.3

1. 設定ファイルをプロジェクト内の任意の場所に作成

```
[project]$ mkdir -p signal/out
[project]$ touch signal/signal.config
```

2. ファイルの記載(引用符は使えません)※

```
[SCAN]
llm_key=YOUR_KEY
cache_file=signal/cache.db
summary_report_file=signal/out/scan-summary.txt
log_file=signal/out/scan.log
error_log_file=signal/out/signal_errors.json
export_results=signal/results_cache.json
exclude_paths=examples/,tests/
oversight_embedding_enabled=True
#oversight=False
#dataflow=False
exploit=True
#extra_extensions=.hs # Haskell ファイルを認識させる
```

Note

signal -help でオプションの一覧を見ることができます。「-」を「_」に置き換えます

1.3.5.2 2.5.2. Bridge CLIの設定ファイルの記載

ブラック・ダックのLLMのURL、発行されたLLM Gatewayキー、Polarisアクセストークンを環境変数に設定します。Bridge CLIの設定ファイルにPolarisとBlack Duck Signalの項目を記載します

1. ブラック・ダックのLLMのURL、発行されたLLM Gatewayキー、Polarisアクセストークンを環境変数に設定する

```
export BRIDGE_SIGNAL_LLM_URL=https://llm.core.blackduck.com
export BRIDGE_SIGNAL_LLM_KEY="YOUR_LLM_KEY"
export BRIDGE_POLARIS_ACCESS_TOKEN="YOUR_TOKEN"
```

2. Bridge CLIのJSON設定ファイルを作成し、アセスメントタイプをAIに設定する。Black Duck Signalの設定ファイルとバイナリの場所を設定する

```
{
  "data": {
    "polaris": {
      "application": {
        "name": "YourApp"
      },
      "project": {
        "name": "your-project"
      },
      "branch": {
        "name": "main"
      },
      "assessment": {
+       "types": ["AI"]
      },
      "serverUrl": "https://poc.polaris.blackduck.com/"
    },
+   "signal": {
+     "args": "--config=signal/signal.config",
+     "execution": {
+       "path": "path/to/signal-pyarmor-linux-x86_64"
+     }
+   }
  }
}
```

1.3.6 2.6. Bridge CLIの実行

設定ファイルを指定して通常のBridge CLIを用いたPolarisでの解析実行時と同じように実行します

1.3.6.1 ■ Bridge CLIの実行例

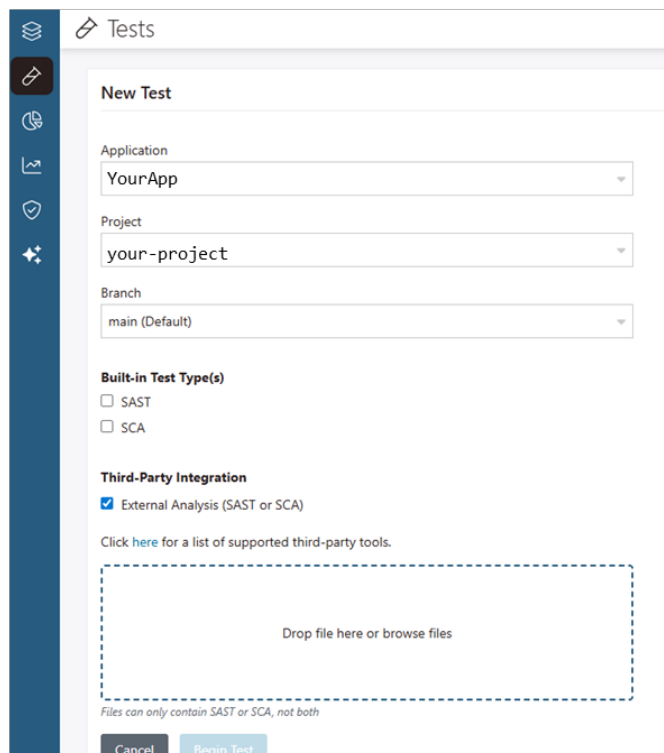
```
[your-project]$ bridge-cli --stage polaris --input your/polaris_config.json
..(omit)..
2025-12-16 23:57:50.4535 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-16 23:57:50.4543 UTC [Polaris AI-Scan] INFO: INFO:__main__:# SINGLE-FILE ANALYSIS
2025-12-16 23:57:50.4544 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-16 23:57:50.4711 UTC [Polaris AI-Scan] INFO: INFO:__main__:Identified 1378 files for analysis
2025-12-16 23:57:50.4712 UTC [Polaris AI-Scan] INFO: INFO:__main__:Default analysis enabled
2025-12-16 23:57:50.8172 UTC [Polaris AI-Scan] INFO: INFO:utils:Analyzing 1 of 1378 (0%) # 解析が始まります
..(omit)..
2025-12-17 02:11:44.0058 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-17 02:11:44.0058 UTC [Polaris AI-Scan] INFO: INFO:__main__:# DATAFLOW AGENT
2025-12-17 02:11:44.0058 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-17 02:11:44.0058 UTC [Polaris AI-Scan] INFO: INFO:__main__: Sending xxe_vulnerability to dataflow analysis age
..(omit)..
2025-12-17 02:11:44.0560 UTC [Polaris AI-Scan] INFO: INFO:agents.sast:Running 283 tasks...
..(omit)..
2025-12-17 02:28:40.0906 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-17 02:28:40.0907 UTC [Polaris AI-Scan] INFO: INFO:__main__:# OVERSIGHT AGENT
2025-12-17 02:28:40.0907 UTC [Polaris AI-Scan] INFO: INFO:__main__:#####
2025-12-17 02:28:40.0907 UTC [Polaris AI-Scan] INFO: INFO:__main__:Performing additional triage for each result...
```

SARIFファイルが`.bridge/polaris_ai_scan/` フォルダに作成されます

1.3.7 2.7. 解析結果の確認

Polaris UI上のTestsタブから解析結果のSARIFファイルをインポートします。対象プロジェクトのIssueタブでツールの種類がExternal AnalysisとなっているのがBlack Duck Signalの結果です

1. PolarisのTestsタブからアプリケーションとプロジェクトを選んでSARIFファイルをインポートします



2. Polaris UI上の検知結果を確認します(SAST – External Analysis)

[Summary](#)
[Issues](#)
[Components](#)
[Licenses](#)
[Tests](#)
[Branches](#)
[Settings](#)

☐
☐
☐

H

H

H

Integer Overflow / Und...

Divide By Zero

NULL Pointer Dereferen...

src/Core/memory_man...

src/autograd/loss_funct...

src/tensor_manipulatio...

memory_manage...

loss_functions.c

tensor_manipulat...

SAST - External...

SAST - External Analysis

SAST - External...

Not Triaged

Not Triaged

Not Triaged

CWE-190

CWE-369

CWE-476

[Issue Details](#)
[Contributing Code Events](#)
Severity: H **Issue Type:** Divide By Zero

Location: src/autograd/loss_functions.c

1. User-supplied input tensor is processed and the number of elements (n) is computed from its dimensions.
2. Division operation using n results in a division by zero when n equals zero (i.e. when the tensors are empty).
3. The data is untrusted because the tensor dimensions (specifically the number of elements, n) are derived from user-supplied input tensors. If both the input and target tensors are empty, the division operation will result in a division by zero.
4. Add validation to check if n is zero before performing division and handle the empty tensor case appropriately.

```

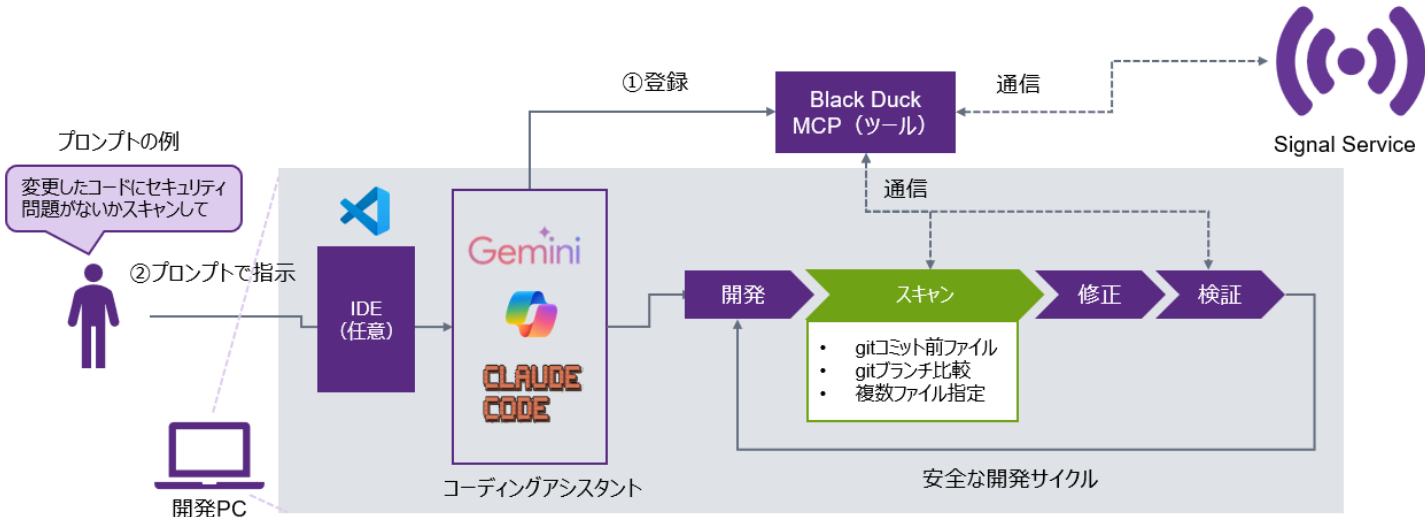
205  n = tensor_numel(input);
...
if (n == 0) {
    tensor_set_float(result, 0, 0.0f);
} else {
    tensor_set_float(result, 0, mae / (float)n);

```

1.4 3. ユースケースII | コーディングアシスタントと共に使う

1.4.1 3.1. 解析フロー

利用中のコーディング・アシスタントにBlack Duck MCPを登録することでコードの差分解析を活用しながら安全に開発サイクルを回すことができます



Analysis Flow for MCP Server

1.4.2 3.2. 解析モード

コードの変更分を解析するモードとファイルを指定して解析するモードがあります

モード		概要	ユースケース
run_diff_security_scan	all-uncommitted	コミットされていない全てのdiffレベルの変更(staged + unstaged)を解析します	• 現在のブランチの解析 • フィーチャーブランチの解析
	reference-branch	mainまたはmasterブランチから乖離した全てのdiffレベルの変更を解析します	
run_security_scan		ファイル全体を解析します	特定のファイルセットの解析

1.4.3 3.3. 共通準備

1.4.3.1 3.3.1. npmのインストール

Node.jsをインストールし、ブラック・ダックのレジストリを登録します

- 1. Node.js([URL](#))のインストラクション(右図参照)に従ってnpmをインストールする

Linux 用のNode.js@ v24.12.0 (LTS) と npm を nvm

を使ってダウンロードする

補足 すぐに最新の機能を試したいですか？最新バージョンのNode.jsを代わりにインストールして最新の機能を試してください！

```
1 # nvmをダウンロードしてインストールする：
2 curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
3
4 # シェルを再起動する代わりに実行する
5 \. "$HOME/.nvm/nvm.sh"
6
7 # Node.jsをダウンロードしてインストールする：
8 nvm install 24
9
10 # Node.jsのバージョンを確認する：
11 node -v # "v24.12.0"が表示される。
12
13 # npmのバージョンを確認する：
14 npm -v # "11.6.2"が表示される。
```

Bash クリップボードにコピー

2. Node.jsをPATHに登録する※

```
export NODE_PATH=${HOME}/.nvm/versions/node/v24.12.0/bin
export PATH=${PATH}:${NODE_PATH}
```

Note

※Node.jsのバージョンは自身の環境に合わせて適宜変更してください

1.4.3.2 3.3.2. 設定ファイルの作成

Black Duck Signalの実行可能ファイルと同じ場所に、scan-configというファイルを作成することで実行時に設定が読み込まれます

1.4.3.2.1 ■ 設定ファイルの概要

- 目的:実行時に共通設定を読み込む
- 要件:Black Duck Signalのバージョン >= 0.1.3
- 使い方:Black Duck Signalの実行可能ファイルと同じ場所に「.scan-config」という名前のファイルを作成(下図参照)

1. 設定ファイルをBlack Duck Signalの実行可能ファイルと同じ場所に作成

```
# For MCP server
$ touch ~/.blackduck/mcp/signal/.scan-config
```

2. ファイルの記載(除外ファイルの設定の例)

```
[SCAN]
oversight_embedding_enabled=True
exclude_paths=examples/,tests/
#include_paths=path1,path2
#llm_key=<secret>
#oversight=False
#dataflow=False
```

1.4.4 3.4.A. Visual Studio CodeのCopilotと共に使う

1.4.4.1 3.4.A.1. 実行までの流れ

Visual Studio Code上でBlack Duck MCPを登録するだけで使えます

1. 準備

LLM Gateway用のキーの作成(事前にいただいた情報をもとにブラック・ダックが発行いたします)

2. 設定

Black Duck MCPを登録(Visual Studio Code上の操作)

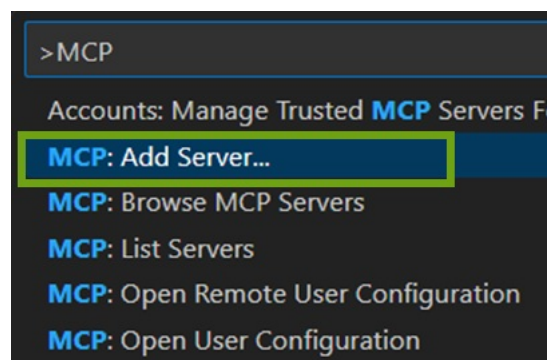
3. 実行

Copilotを使ってファイルを解析する

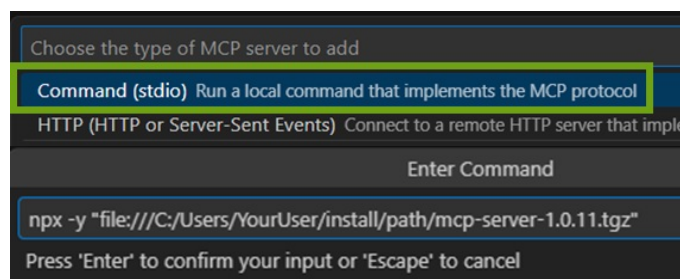
1.4.4.2 3.4.A.2. 設定(1/2) | ブラック・ダックのMCPサーバーの追加

コマンドパレットからMCPサーバーの追加を選択し、コマンドを入力します。スコープはGlobalに設定してVisual Studio Code上の全てのプロジェクトからお渡ししたブラック・ダックのMCPサーバーが使えるようにします

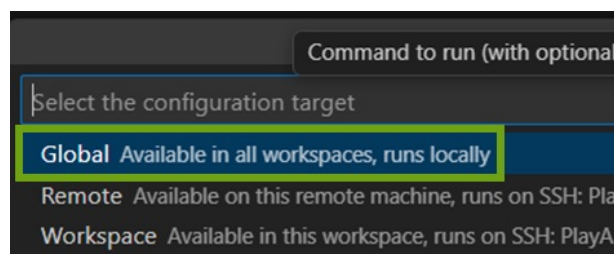
1. Ctrl+Shift+Pでコマンドパレットを開き、「MCP: Add server」を選択※1



2. Command (stdio)を選択し、お渡ししたMCPサーバーのファイルを指定して「npx -y file://[file path]」と入力



3. スコープとしてGlobalを選択※2



Note

1. Visual Studio CodeのRemote DevelopmentでVMに接続している場合は『Appendix B. リモート環境にMCPサーバーを追加する』を参照してください
2. Globalの場合、当該のMCPサーバーをVisual Studio Code上の全てのプロジェクトから使えます。Workspaceを選ぶと現在のWorkspaceからしか使えません

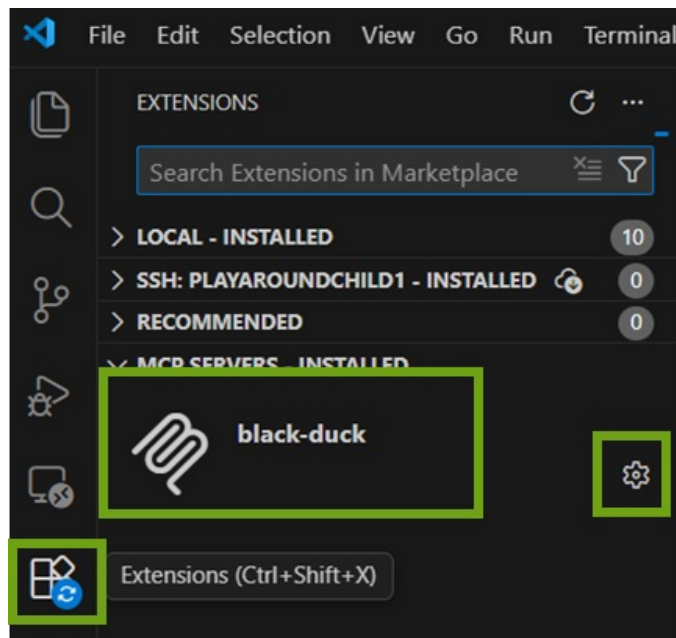
1.4.4.3 3.4.A.3. 設定(1/2) | LLM Gatewayキーの設定ファイルへの追加

MCPサーバー追加直後に開いたmcp.jsonのargs項目の後ろに環境変数として発行されたLLM Gatewayキーを記載します

1. MCPサーバー追加後に開いたmcp.jsonのargs項目の後ろにLLM Gatewayキーの設定を追加します

```
{
  "servers": {
    "black-duck": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "file:///C:/Users/YourUser/insall/path/mcp-server-1.1.x.tgz"
      ],
+   "env": {
+     "BLACKDUCK_MCP_GATEWAY_KEY": "YOUR_LLM_KEY"
+   }
    }
  }
}
```

2. ExtensionタブのMCP SERVERSに現れるMCPサーバーを設定から起動します(ログの例は下)



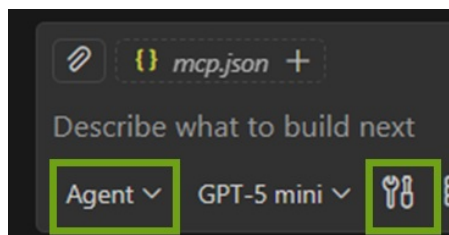
```
2025-12-28 16:05:20.436 [info] Waiting for server to respond to `initialize` request...
2025-12-28 16:05:21.576 [info] Discovered 1 tools
```

1.4.4.4 3.4.A.4. 実行(1/4) | ブラック・ダックのMCPサーバーをエージェントモードで有効化する

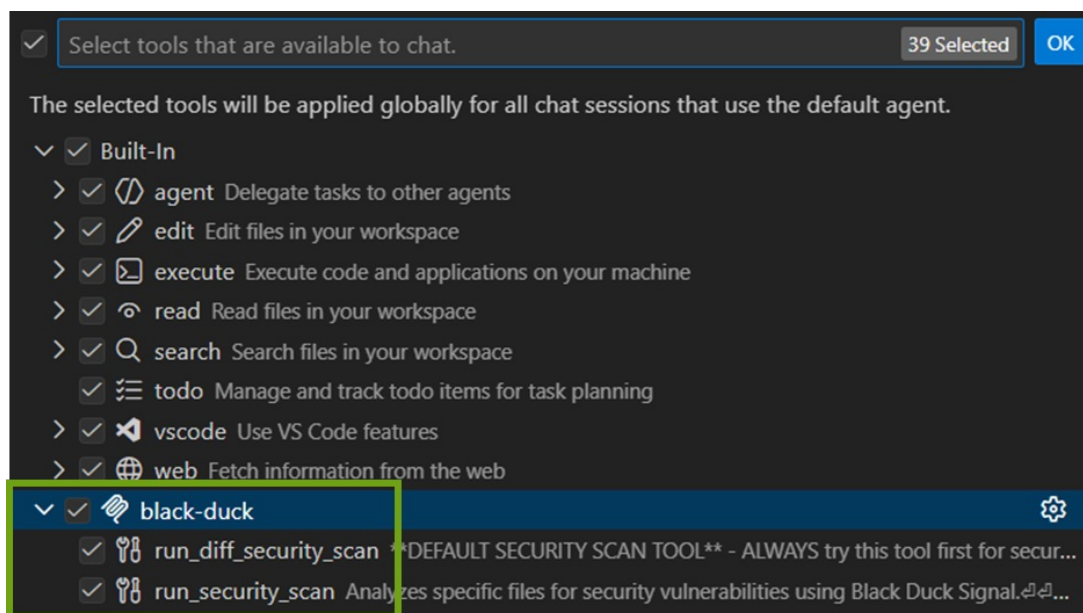
Visual Studio Codeの右のチャット窓をエージェントモードにし、設定から追加したブラック・ダックのMCPサーバーを有効化します

1. 右上からチャット窓を開き、エージェントモードであることを確認して設定を開きます





2. 追加したMCPサーバーにチェックを入れて有効化します



1.4.4.5 3.4.A.5. 実行(2/4) | git-diffによる差分解析

Black Duck Signalはデフォルトではgit-diffを使った差分解析を実行します。コードを変更し、変更分の解析を指示します。セキュリティスキャン時に裏でBlack Duck Signalが使われる保証はないため、明示的に利用を指示します

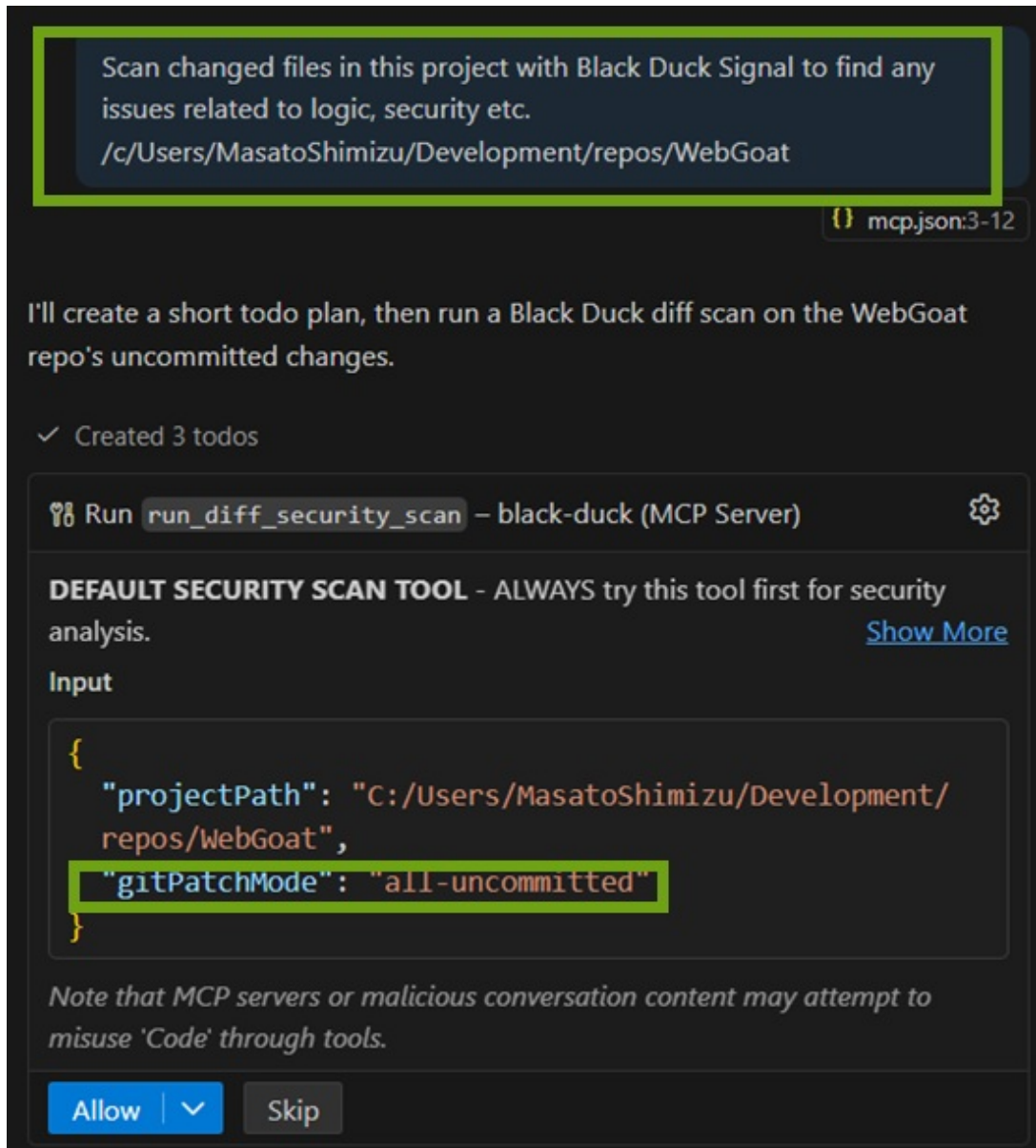
1. コードを変更します

```
AzureAD+MasatoShimizu@BD-97650 MINGW64 ~/Development/repos/WebGoat (bd_main)
$ git status
On branch bd_main
Your branch is ahead of 'blackduck/bd_main' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   VULNERABLE_ACHIEVEMENTS.md
        modified:   coverity.conf
        modified:   coverity/coverity.yaml
        modified:   src/main/java/org/owasp/webgoat/lessons/jwt/JWTQuiz.java
        modified:   src/main/java/org/owasp/webgoat/server/StartWebGoat.java
        modified:   src/main/resources/static/vulnerable-achievements.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .blackduck/
        blackduck-detect/
        polaris/
        seeker/
        src/main/java/org/owasp/webgoat/container/achievements/ShadowAchievementAPI.java
```

2. Black Duck Signalの使用を明記して変更分の解析を指示します



コミットしていないコードに対して解析が行われる(下段)

1.4.4.6 3.4.A.6. 実行(3/4) | ブランチ参照による差分解析

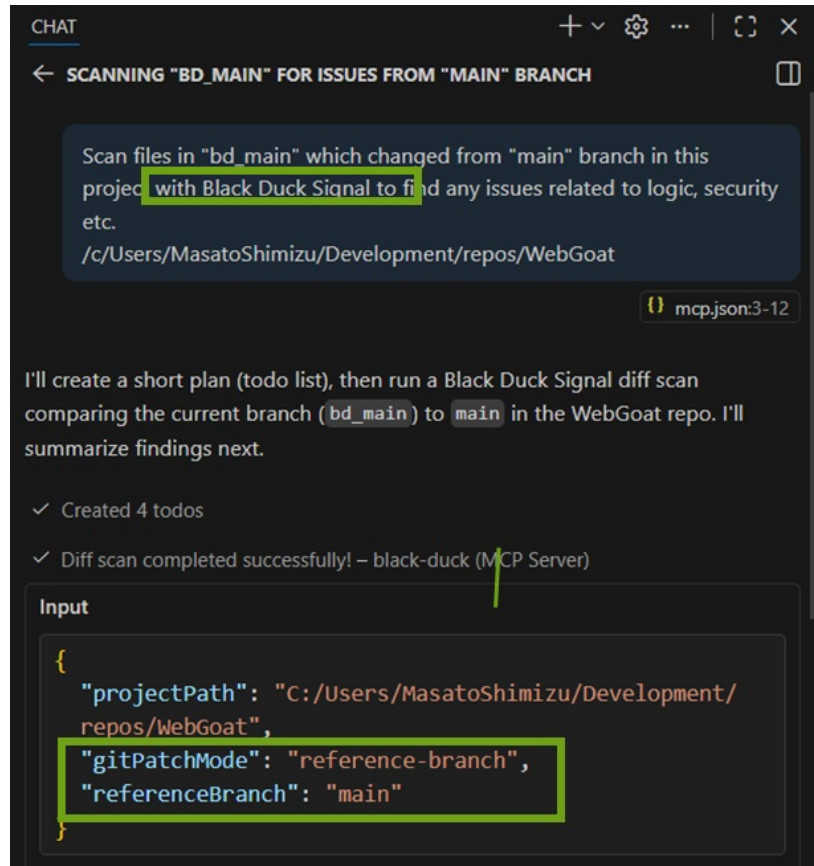
Black Duck Signalはデフォルトではgit-diffを使った差分解析を実行します。コードを変更し、変更分の解析を指示します。セキュリティスキャン時に裏でBlack Duck Signalが使われる保証はないため、明示的に利用を指示します

1. コードを変更します

```
AzureAD+MasatoShimizu@BD-97650 MINGW64 ~/Development/repos/WebGoat (bd_main)
$ git log --oneline
43d57462 (HEAD -> bd_main) Add secret
2e4fe8e3 (blackduck/bd_main, blackduck/HEAD) Add vulnerability to achievements container
452961b2 (original) Add leaderboard
d4238ab4 (origin/main, origin/HEAD, main) chore: bump alpine from 3.20 to 3.22 (#2239)
35d369b8 chore: bump org.apache.commons:commons-text from 1.13.1 to 1.14.0 (#2241)
6f1370a0 chore: bump devops-infra/action-pull-request from 0.6.1 to 1.0.2 (#2248)
```

mainブランチからの変更分(下段)

2. Black Duck Signalの使用を明記して変更分の解析を指示します



指定したブランチが参照先として使われている(下段)

1.4.4.7 3.4.A.7. 実行(4/4) | 複数ファイルの解析

Black Duck Signalは複数のファイルを指定して解析できます。全体の解析を行う際はフォルダ内のファイルを全て選択するようにプロンプトで指示します

1.4.4.7.1 ■ 特定のファイルを指定して解析を指示

Scan these files in the project below with Black Duck Signal to find any issues related to logic, vulnerability etc.

Project: C:/Users/MasatoShimizu/Development/repos/WebGoat

Files:

- src/main/java/org/owasp/webgoat/container/achievements/Achievement.java
- src/main/java/org/owasp/webgoat/container/achievements/AchievementController.java

I'll create a short todo plan, then run Black Duck Signal on the specific files you listed.

✓ Created 3 todos

 Run `run_security_scan` – black-duck (MCP Server) 

Analyzes specific files for security vulnerabilities using Black Duck Signal. [Show More](#)

WHEN TO USE:

Input

```
{
  "projectPath": "C:/Users/MasatoShimizu/Development/repos/WebGoat",
  "filePaths": [
    "C:/Users/MasatoShimizu/Development/repos/WebGoat/src/main/java/org/owasp/webgoat/container/achievements/Achievement.java",
    "C:/Users/MasatoShimizu/Development/repos/WebGoat/src/main/java/org/owasp/webgoat/container/achievements/AchievementController.java",
  ]
}
```

特定のファイル群を指定(上段)、ファイル解析が行われている(下段)

1.4.4.7.2 ■ 特定のフォルダ内の全てのファイルを解析するように指示



フォルダを指定(上段)、ファイル解析として扱われる(下段)

1.4.5 3.4.B. Visual Studio CodeのClaudeと共に使う

1.4.5.1 3.4.B.1. 実行までの流れ

Claude CodeからMCPサーバーを使用するにはMCPサーバーのローカルインストールが必要です。Claude CodeのVisual Studio Codeのプラグイン側の設定も必要になります

1. 準備

- LLM Gateway用のキーの作成(事前にいただいた情報をもとにブラック・ダックが発行いたします)
- Black Duck MCPのローカルインストール(共通準備で完了済)
- Claude Codeのインストール
- IDEにClaude Codeプラグインをインストールする

2. 設定

- ClaudeにBlack Duck MCPを登録(CLI)
- Claude Codeの実行用の環境変数の設定
- Visual Studio Codeプラグインの設定

3. 実行

Claude Codeを使ってファイルを解析する

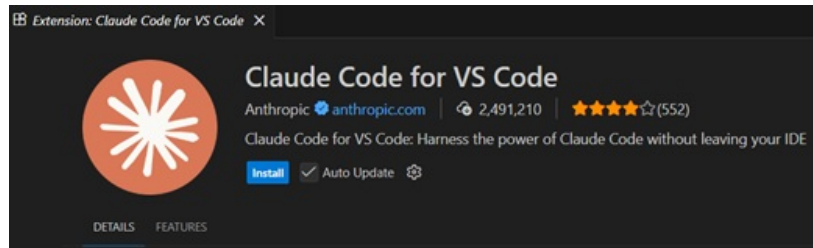
1.4.5.2 3.4.B.2. 準備 | Claude CodeおよびIDEプラグインのインストール

Claudeのインストラクションに従ってClaude Codeをインストールします。Claude CodeのIDEプラグインもインストールし、ログインプロンプトをオフにします

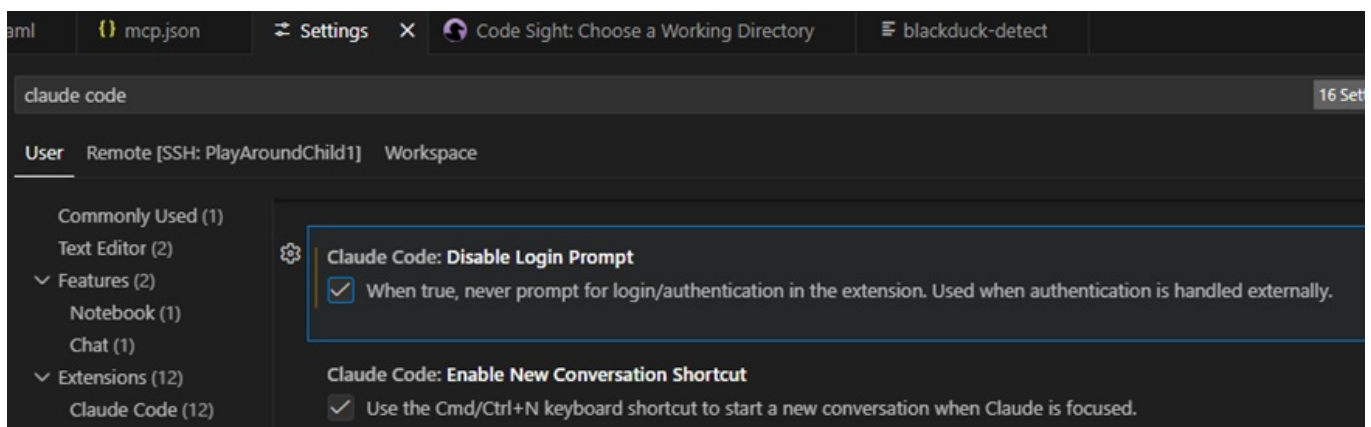
1. Claudeの[インストラクション](#)に従って使ってClaude Codeをインストール(以下はLinuxの例)

```
curl -fsSL https://claude.ai/install.sh | bash
```

2. IDE(ここではVisual Studio Code)プラグインをインストールします



3. 設定からClaudeのログインプロンプトをオフにします



1.4.5.3 3.4.B.3. 設定(1/2) | MCPサーバーの登録

準備段階で作成したLLM Gatewayキーとブラック・ダックのLLMのURLをClaude Codeの環境変数として設定し、ClaudeのMCPに弊社からお渡ししたBlack Duck MCPサーバーを登録します

1. 作成したLLM Gatewayキーを使い、Claudeにお渡ししたBlack Duck MCPサーバーのファイルを登録します※

```
$ claude mcp add --transport stdio black-duck --scope user --env  
BLACKDUCK_MCP_GATEWAY_KEY="YOUR_LLM_KEY" -- npx -y "file:///path/to/mcp-server-1.1.x.tgz"
```

```
Added stdio MCP server black-duck with command: mcp-server to user config  
File modified: /home/your-user/.claude.json
```

2. 接続できているか確認します

```
$ claude mcp list  
Checking MCP server health...
```

```
black-duck: mcp-server - ✓ Connected
```

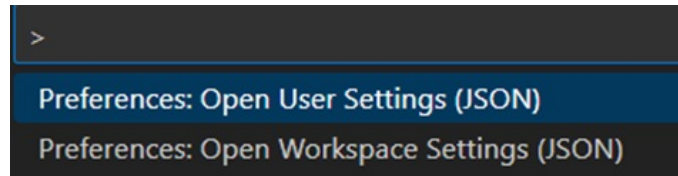
Note

- スcopeはuserになっており、単一ユーザーでどのプロジェクトでも使える設定です
- Windowsでは「file:///C:/Users/YourUser/path/mcp-server-1.0.11.tgz」のようになります

1.4.5.4 3.4.B.4. 設定(2/2) | Visual Studio Codeプラグインの設定

Claude CodeのVisual Studio Codeプラグインのユーザースコープの設定ファイルにブラック・ダックのLLMゲートウェイを使わせる環境変数、ログインプロンプトをオフにするオプションを追加し、モデルもclaude-sonnet-4を指定します

1. コマンドパレット(Ctrl + Shift + P)からユーザーのsetting.jsonを開きます※



2. ブラック・ダックのLLMゲートウェイを使うための環境変数、ログインプロンプトをオフにするオプション、claude-sonnet-4を指定するオプションを追加します

```
+  "claudeCode.environmentVariables": [  
+    {  
+      "name": "ANTHROPIC_AUTH_TOKEN",  
+      "value": "YOUR_LLM_KEY"  
+    },  
+    {  
+      "name": "ANTHROPIC_BASE_URL",  
+      "value": "https://llm.core.blackduck.com"  
+    }  
+  ],  
+  "chat.viewSessions.orientation": "stacked",  
+  "claudeCode.disableLoginPrompt": true,  
+  "claudeCode.selectedModel": "claude-sonnet-4",
```

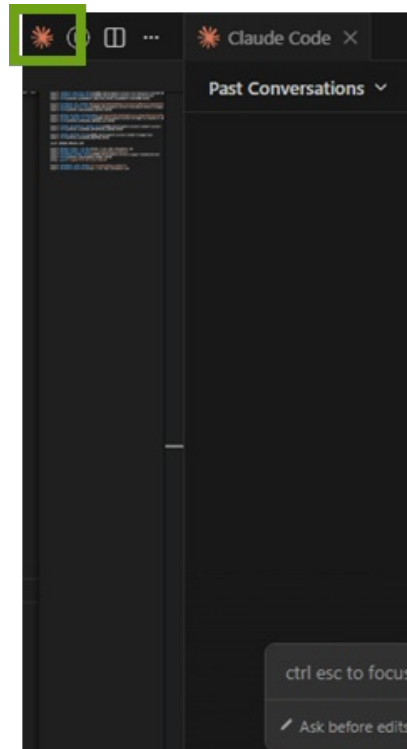
Note

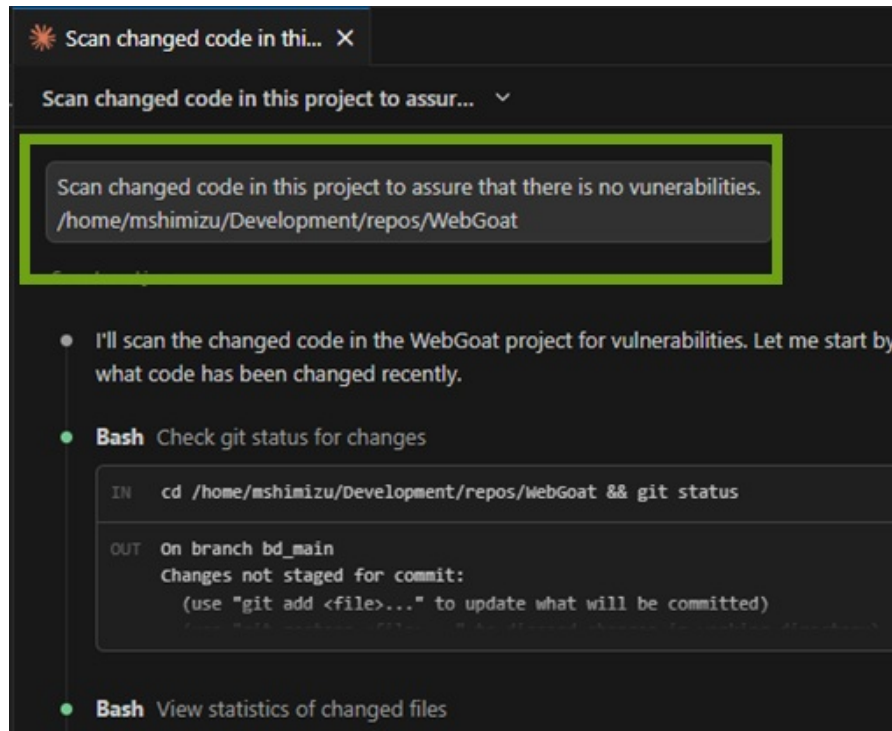
※Workspaceのsetting.jsonにClaudeのオプションを記載しない前提

1.4.5.5 3.4.B.5. 実行(1/2) | 指示の失敗例

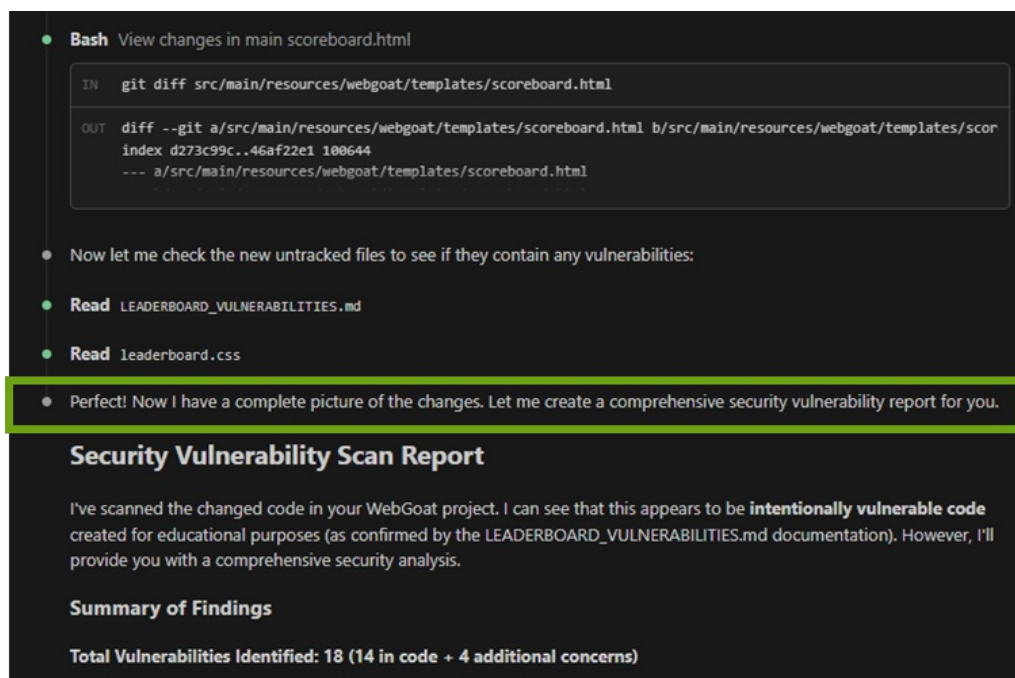
明示的にBlack Duck Signalを使うよう指示しないと使われないことがあります

1. Claudeアイコンからプラグインを起動し、変更したファイルの解析を指示※





2. 結果が表示されます



Note

WindowsにおいてはファイルパスはWindowsのネイティブ書式しかサポートされません(GitBash書式不可)

1.4.5.6 3.4.B.6. 実行(2/2) | 指示の成功例

Claude Codeから変更したファイルの解析を依頼します。明示的にBlack Duck Signalの使用を指示します

1. Black Duck Signalの利用を明記してセキュリティ・スキャンを指示※

```
Scan all files under the "src" directory of this project:
C:\Users\MasatoShimizu\Development\repos\legacy-insecure-bank
Use Black Duck Signal specific file analysis to check if there is any issue of logic, security etc..

If Black Duck Signal returns Attack Chain information, write it to console.

c:\Users\MasatoShimizu\blackduck\mcp\logs\black-duck-mcp.log

• Thinking ~
• I'll help you scan the "src" directory of your legacy-insecure-bank project using Black Duck Signal
```

```
• Perfect! I can see there are 4 COBOL files in the src directory. Now let me try the Black Duck Signal security scan. I'll first
work because it's not a git repository or there are no changes, I'll fall back to scanning the specific files.

• Black-duck [run_diff_security_scan]

OUT The user doesn't want to proceed with this tool use. The tool use was rejected (eg. if it was a file edit

• I understand - you want me to scan the specific files. Let me use the file-specific Black Duck scan tool to analyze all the

• Black-duck [run_security_scan]

OUT {"sarifFilePath":"","status":"failure","resultMessage":"Input validation failed: filePaths is required and
```

デフォルトのgit-diffスキャンでBlack Duck Signalが使われている(上段)、拒否して特定のファイルを解析するよう指示できる(下段)

2. 結果が表示されます

```
• Excellent! Black Duck Signal has completed the security analysis and found multiple critical security issues. Let me e

Black Duck Signal Security Analysis Results

• CRITICAL SECURITY ISSUES FOUND

The scan detected 9 security vulnerabilities (5 errors, 4 warnings) in your COBOL banking application:

Summary by Severity:

• Critical: SQL Injection (CVSS 9.8)
• High: Integer Overflow/Underflow (CVSS 8.2), Hard-coded Credentials (CVSS 8.0)
• Medium: Multiple Input Validation issues (CVSS 6.5), Information Disclosure (CVSS 6.0)
• Low: Automatic Upgrade Disabled (CVSS 4.0)

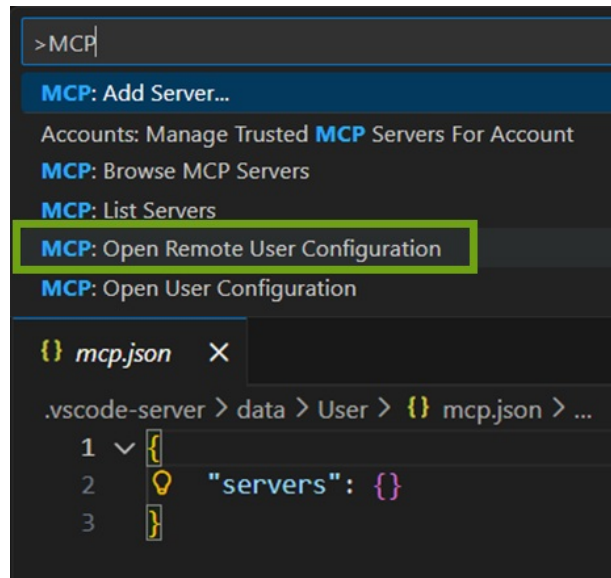
⚠ Attack Chain Analysis:

Black Duck Signal identified a critical attack chain:
```

1.5 Appendix. リモート環境にMCPサーバーを追加する

コマンドパレットからリモート用のMCPの設定ファイルを開き、それを編集してMCPサーバーを追加します。追加されたMCPサーバーはExtensionsタブを開くとリモートアイコン付きで表示されます

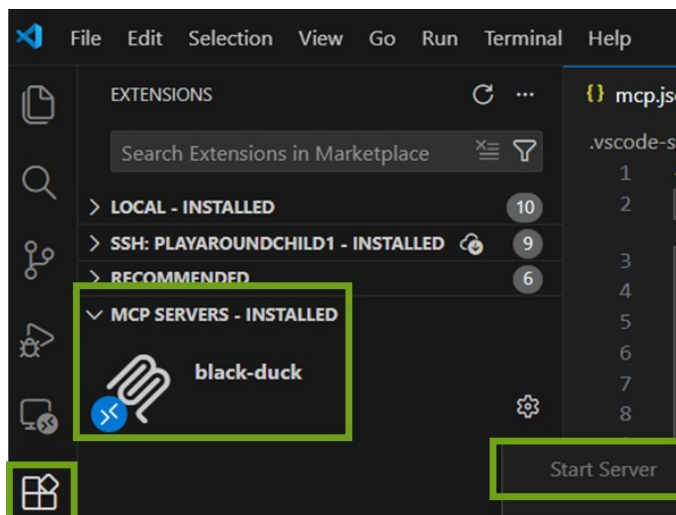
1. 「Open Remote User Configuration」を選択し、リモートの設定ファイルを開く



2. 開いた設定ファイルを編集して環境変数と共にブラック・ダックのMCPサーバーを追加する

```
{
  "servers": {
    "black-duck": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "file:///path/to/mcp-server-1.1.x.tgz"
      ],
      "env": {
        "BLACKDUCK_MCP_GATEWAY_KEY": "YOUR_LLM_KEY"
      }
    }
  }
}
```

3. ExtensionタブのMCP SERVERSに現れるMCPサーバーを起動します





本資料のご使用について

- 本資料の内容は予告なく変更することがありますのでご了承ください。
- 本資料の作成者および提供者は、本資料に関していかなる種類の保証を負うものではありません。また本資料の作成者および提供者は、本資料の使用における直接的・間接的また結果的に生じたいかなる損害についても責任を負うものではありません。
- 本資料の内容は、本資料の作成者および権利所有者に帰属します。使用者はこれらの権利を尊重し、無断での利用や転載を行わないようにお願いします。